

Rapport de Projet – SAE 2.02 : Vivez l'aventure !

Timothy Maume, Neven Quefelec, Eliott Tauzi, Guewen Monlouis

Présentation de l'application

Notre application modélise et résout informatiquement la structure d'un livre-jeu en respectant les spécifications du cahier des charges. Au niveau de l'interface, c'est une application s'exécutant intégralement en ligne de commande.

Concernant la mécanique de jeu, les énigmes présentes sur les pages sont affichées pour l'immersion du joueur, et les objets nécessaires sont automatiquement collectés lors de l'arrivée sur la page correspondante. Afin de répondre à la consigne demandant la génération d'énigmes, nous avons intégré la librairie externe **Lorem Ipsum** (fichier `.jar`). Cet outil génère le contenu textuel des pages, pour répondre à la demande d'auto-génération sans nécessiter de rédaction à la main. Source : <https://loremipsum.sourceforge.net/>

Justification des choix et des algorithmes

Pour la recherche de solution, notre application s'appuie sur deux algorithmes classiques : le **Parcours en Largeur (BFS)** et l'algorithme de **Dijkstra**.

Justification : Ces algorithmes répondent points important pour créer un chemin :

- Le **BFS** permet de trouver de manière optimale la *longueur d'une solution* (le chemin minimisant le nombre de pages traversées).
- **Dijkstra** est utilisé pour optimiser le *temps de parcours* (la somme des temps de résolution ou de la difficulté des énigmes), modélisé ici par des arêtes pondérées.

Correction et terminaison : La garantie que nos algorithmes trouvent toujours une solution (sans boucler à l'infini et sans rester bloqués) repose sur le graph généré. Conformément aux contraintes du sujet, le graphe est construit de sorte qu'il est possible d'accéder à la sortie depuis n'importe quelle page. Concrètement, notre générateur s'assure de l'absence totale de cul-de-sac (chaque nœud possède 2 à 3 arêtes sortantes). La seule boucle présente est le retour à la page de départ pour éviter les impasses. Les algorithmes n'y passent pas car ils ont toujours moyen d'arriver à la sortie autrement.

Génération automatique du livre-jeu

Notre module génère automatiquement des livres-jeux (pages, texte Lorem Ipsum, objets et énigmes) en respectant des contraintes topologiques strictes : chaque page propose 2 à 3 choix pour éviter les culs-de-sac, et la sortie reste accessible depuis n'importe quel point.

Pour optimiser cette création, nous avons développé et comparé deux algorithmes :

1. **Le "Graphe Simple"** : Il trace d'abord un chemin valide garanti (reliant le départ, les objets requis, puis la fin). Ensuite, il ajoute aléatoirement 2 à 3 arêtes aux autres pages pour créer des détours.
2. **Le "Graphe Aléatoire"** : Il relie d'abord toutes les pages de manière purement aléatoire. Une passe de correction est ensuite appliquée pour vérifier la connexité : si certaines pages ou la sortie sont inaccessibles, l'algorithme ajoute les arêtes nécessaires pour débloquer le jeu.

Comparaison : Le **Graphe Simple** est plus performant en temps d'exécution, car sa solution est garantie dès le départ sans nécessiter de lourdes vérifications. À l'inverse, le **Graphe Aléatoire** est légèrement plus lent à générer à cause de sa phase de correction *a posteriori*, mais il offre une architecture beaucoup plus chaotique et imprévisible pour le joueur.

Tests

Nous avons mis en place un protocole pour comparer les performances du BFS et de Dijkstra sur la recherche de solutions.

Protocole :

- **Données de test** : Des livres-jeu ont été générés aléatoirement, avec des tailles variant de 5 à 20 pages.
- **Pondération (Temps de parcours)** : Pour tester l'algorithme de Dijkstra, le temps de résolution des énigmes de chaque page a été généré aléatoirement sur une large plage de valeurs, de 0 à 1000.
- **Métriques** : Nous avons comparé la longueur de la solution, le temps de parcours total et le temps d'exécution.

Analyse des résultats : Sur l'ensemble des graphes générés, les temps d'exécution des deux algorithmes ont été presque nuls pour des volumes allant jusqu'à 20 pages. L'intérêt de la comparaison c'est le chemin trouvé. L'utilisation d'une grande plage de valeur dans les pondérations (0 à 1000) démontre que le BFS, même si il est efficace pour minimiser la longueur (le nombre de pages), propose des chemins dont le temps de parcours n'est pas optimal. À l'inverse, Dijkstra trouve des chemins qui nécessitent de traverser plus de pages, mais dont le score cumulé de difficulté (temps de résolution) est inférieur à celui du BFS.